

SO-101 / LeRobot

コマンド集 カメラ・Hugging Face 版

2026-10-08 時点

構成

今回の固定構成: Camera 0 = 手元 (hand)、Camera 1 = 俯瞰 (top)、Hugging Face = Pinset。ポート番号は接続し直すと変わる場合があるため、収録前に確認する。

現在の設定

項目	設定
Hugging Face	Pinset
Follower port	/dev/tty.usbmodem5B7B0139951
Leader port	/dev/tty.usbmodem5B7B0140731
Follower ID	my_awesome_follower_arm
Leader ID	my_awesome_leader_arm
Camera 0	hand (手元)
Camera 1	top (俯瞰)
Dataset	Pinset/so101_pick_place_2cam_100_20261008_140403
1000-step policy	Pinset/act_so101_pick_place_2cam_1000

1. Conda 環境を起動

```
conda activate lerobot
```

LeRobot のリポジトリへ移動する場合:

```
cd ~/lerobot
```

2. Hugging Face のログイン確認

```
hf auth whoami
```

Pinset になっていれば OK:

```
✓ Logged in  
user: Pinset
```

別アカウントから Pinset へ切り替える場合:

```
hf auth login --force  
hf auth whoami
```

ここで自分の場合

メールアドレス : keita.y2004@gmail.com

パスワード : Pinset2026!

3. SO-101 の USB ポート確認

```
lerobot-find-port
```

注意

USB を抜き差しした後はポート番号が変わる場合がある。Follower / Leader の番号を再確認してから実行する。

4. カメラ確認

```
lerobot-find-cameras opencv
```

今回の割り当て:

```
Camera 0 -> hand (手元カメラ)  
Camera 1 -> top (俯瞰カメラ)
```

5.2 台のカメラ映像だけ確認

```
python - <<'PY'
import cv2

cap0 = cv2.VideoCapture(0)
cap1 = cv2.VideoCapture(1)

for cap in (cap0, cap1):
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    cap.set(cv2.CAP_PROP_FPS, 30)

while True:
    ret0, frame0 = cap0.read()
    ret1, frame1 = cap1.read()

    if ret0:
        cv2.imshow("Camera 0 - hand", frame0)
    if ret1:
        cv2.imshow("Camera 1 - top", frame1)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap0.release()
cap1.release()
cv2.destroyAllWindows()
PY
```

終了は映像ウィンドウを選択して **control+C**。

6. Leader -> Follower テレオペ

```
lerobot-teleoperate \
--robot.type=so101_follower \
--robot.port=/dev/tty.usbmodem5B7B0139951 \
--robot.id=my_awesome_follower_arm \
--teleop.type=so101_leader \
--teleop.port=/dev/tty.usbmodem5B7B0140731 \
--teleop.id=my_awesome_leader_arm
```

Follower と **leader** のポート番号は自分のグループのものに書き換える。

(以下、このような **robot.port** が出てきた際も同じ)

7.2 カメラで 100 エピソード収録

手元(hand)と俯瞰(top)を同時に記録し、100 エピソードを Hugging Face へアップロードする。

```
lerobot-record \  
--robot.type=so101_follower \  
--robot.port=/dev/tty.usbmodem5B7B0139951 \  
--robot.id=my_awesome_follower_arm \  
--robot.cameras="{ hand: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, top: {type: opencv,  
index_or_path: 1, width: 640, height: 480, fps: 30}}" \  
--teleop.type=so101_leader \  
--teleop.port=/dev/tty.usbmodem5B7B0140731 \  
--teleop.id=my_awesome_leader_arm \  
--dataset.repo_id=Pinset/so101_pick_place_2cam_100 \  
--dataset.num_episodes=100 \  
--dataset.single_task="Pick up the blue block and put it in the green square" \  
--dataset.push_to_hub=true \  
--display_data=true
```

実行前に 17 収録コマンドの書き換えテンプレートを確認

重要

今回の実際の Dataset repo は日時付きで
Pinset/so101_pick_place_2cam_100_20261008_140
403。学習時はこの正確な repo_id を使用する。

8. ACT を 1000 steps 学習

```
lerobot-train \  
--dataset.repo_id=Pinset/so101_pick_place_2cam_100_20261008_140403 \  
--policy.type=act \  
--policy.device=mps \  
--policy.repo_id=Pinset/act_so101_pick_place_2cam_1000 \  
--policy.push_to_hub=true \  
--output_dir=outputs/train/act_so101_pick_place_2cam_1000 \  
--job_name=act_so101_pick_place_2cam_1000 \  
--batch_size=8 \  
--steps=1000 \  
--save_checkpoint=true \  
--save_freq=1000 \  
--wandb.enable=false
```

実行前に 18ACT 学習の書き換えテンプレートを確認

補足

--save_freq=1000 は「1000 step ごとにチェック
ポイントを保存」の意味。steps=1000 の場合は
最終時点で保存される。step 数が学習回数とな
る。

9. 長時間学習中に Mac をスリープさせない

学習とは別のターミナルで実行:

```
caffeinate -i
```

終了は Control + C。電源アダプタ接続・蓋を開けたままの運用を推奨。

10. 学習済みモデルを SO-101 で実行

```
lerobot-rollout \
--strategy.type=base \
--robot.type=so101_follower \
--robot.port=/dev/tty.usbmodem5B7B0139951 \
--robot.id=my_awesome_follower_arm \
--robot.cameras="{ hand: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, top: {type: opencv,
index_or_path: 1, width: 640, height: 480, fps: 30}}" \
--policy.path=Pinset/act_so101_pick_place_2cam_1000 \
--policy.device=mps \
--task="Pick up the blue block and put it in the green square" \
--duration=60 \
--display_data=true
```

実行前に 19 学習済みモデルを切り替えて実機実行を確認

重要

hand=Camera 0、top=Camera 1 の名前と番号は、学習時と実行時で一致させる。最初の実機テストはすぐ止められる状態で行う。

11. 学習データ内の動画を探す

```
find ~/.cache/huggingface/lerobot/hub \
-path "*so101_pick_place_2cam*" \
-name "*.mp4"
```

2 カメラ Dataset では、主に observation.images.hand と observation.images.top の動画が保存される。

12. Hugging Face Dataset キャッシュを Finder で開く

```
open ~/.cache/huggingface/lerobot/hub
```

今回の Dataset は、datasets--Pinset--so101_pick_place_2cam_100_20261008_140403 のようなキャッシュ名で保存される。

13. ffmpeg を入れる / QuickTime 向けに変換

```
conda install -c conda-forge ffmpeg
ffmpeg -version
```

QuickTime で開けない動画を H.264 へ変換:

```
ffmpeg -i ~/Desktop/top_camera.mp4 \
-c:v libx264 \
-pix_fmt yuv420p \
-movflags +faststart \
~/Desktop/top_camera_quicktime.mp4

open -a "QuickTime Player" ~/Desktop/top_camera_quicktime.mp4
```

14. よく使う 4 コマンド - クイックリファレンス

A. カメラ確認

```
lerobot-find-cameras opencv
```

B. テレオペ

```
lerobot-teleoperate \
--robot.type=so101_follower \
--robot.port=/dev/tty.usbmodem5B7B0139951 \
--robot.id=my_awesome_follower_arm \
--teleop.type=so101_leader \
--teleop.port=/dev/tty.usbmodem5B7B0140731 \
--teleop.id=my_awesome_leader_arm
```

C. ACT 学習

```
lerobot-train \
--dataset.repo_id=Pinset/so101_pick_place_2cam_100_20261008_140403 \
--policy.type=act \
--policy.device=mps \
--policy.repo_id=Pinset/act_so101_pick_place_2cam_1000 \
--policy.push_to_hub=true \
--output_dir=outputs/train/act_so101_pick_place_2cam_1000 \
--job_name=act_so101_pick_place_2cam_1000 \
--batch_size=8 \
--steps=1000 \
--save_checkpoint=true \
--save_freq=1000 \
--wandb.enable=false
```

実行前に 18ACT 学習の書き換えテンプレートを確認

D. 学習済みモデルを実行

```
lerobot-rollout \
--strategy.type=base \
--robot.type=so101_follower \
--robot.port=/dev/tty.usbmodem5B7B0139951 \
--robot.id=my_awesome_follower_arm \
--robot.cameras="{ hand: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, top: {type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30}}" \
--policy.path=Pinset/act_so101_pick_place_2cam_1000 \
--policy.device=mps \
--task="Pick up the blue block and put it in the green square" \
--duration=60 \
--display_data=true
```

実行前に 19 学習済みモデルを切り替えて実機実行を確認

15. トラブル時に最初に確認すること

- Hugging Face が Pinset でログインされているか: `hf auth whoami`
- Follower / Leader の USB ポート番号が変わっていないか: `lerobot-find-port`
- Camera 0 = hand、Camera 1 = top のままか: `lerobot-find-cameras opencv`
- 404 Repository Not Found なら、Dataset / Model の `repo_id` が正確かを確認する。
- Leader で Input voltage error が出た場合は、Leader 側電源・配線・グリッパーモーター周辺を確認する。
- 出力フォルダが既にある場合は `output_dir` を変更するか、既存結果を確認してから再実行する。

流れ

基本フロー: カメラ確認 -> テレオペ -> 収録 -> Hugging Face Dataset -> ACT 学習 -> Hugging Face Model -> Rollout。

16. 別の学習データ・作業に切り替える場合

この追補では、元のコマンド集の「7. 収録」「8. 学習」「10. 実機実行」を別の作業に使い回す方法を示します。赤色の `{{...}}` は実行前に必ず自分の値へ置き換える欄です。`{{ }}` はそのまま入力しません。

最初に覚えること：データセットとモデルは別物

設定	何を指定する？	変更が必要な場面
<code>--dataset.repo_id</code>	学習に使う収録済みデータの場所	別のデータセットで学習するとき
<code>--policy.repo_id</code>	学習後のモデルを公開・保存する名前	新しいモデルとして区別するとき
<code>--policy.path</code>	実機で読み込む学習済みモデル	使うモデルを切り替えるとき
<code>--output_dir / --job_name</code>	PC 上の出力先 / 実験名	学習を別実験として保存するとき
<code>--dataset.single_task / --task</code>	収録 / 実行時の作業内容	物をつかむ対象などを変更するとき

例：データセット A で学習してモデル A を作ったあと、データセット B で学習するなら、最低限「`dataset.repo_id`」「`policy.repo_id`」「`output_dir`」「`job_name`」を確認・変更します。実機で B のモデルを使うなら「`policy.path`」も B に変更します。

変更箇所一覧（元資料のコマンド対応）

項目	元の値（例）	変更条件
Follower USB port	<code>/dev/tty.usbmodem5B7B0139951</code>	USB 番号が変わったとき
Leader USB port	<code>/dev/tty.usbmodem5B7B0140731</code>	USB 番号が変わったとき
カメラインデックス	<code>hand=0 / top=1</code>	カメラの認識順が変わったとき
Dataset repo ID	<code>Pinset/so101_pick_place_2cam_100_20261008_140403</code>	データセットを替えたとき
学習回数	<code>--steps=1000</code>	学習時間を変えるとき
Policy repo ID	<code>Pinset/act_so101_pick_place_2cam_1000</code>	新しいモデルを作るとき
モデル読み込み先	<code>--policy.path=...</code>	実行するモデルを変えるとき

原則そのまま：`--robot.type=so101_follower`、`--teleop.type=so101_leader`、`--policy.type=act`、Mac で MPS を使う場合の `--policy.device=mps`。カメラの名前・台数・解像度が変わる場合は、学習済みモデルとの入力互換性も確認します。

17. 収録コマンドの書き換えテンプレート

新しい物体・作業を収録するときに使用。赤い欄を自分で置き換えて実行します。

```
lerobot-record \  
  --robot.type=so101_follower \  
  --robot.port={{FOLLOWER_PORT}} \  
  --robot.id=my_awesome_follower_arm \  
  --robot.cameras="{hand: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, top: {type: opencv, index_or_path: 1, width: 640, height: 480, fps: 30}}" \  
  --teleop.type=so101_leader \  
  --teleop.port={{LEADER_PORT}} \  
  --teleop.id=my_awesome_leader_arm \  
  --dataset.repo_id={{HF_USER}}/{{NEW_DATASET_NAME}} \  
  --dataset.num_episodes={{EPISODES}} \  
  --dataset.single_task="{{TASK_IN_ENGLISH}}" \  
  --dataset.push_to_hub=true \  
  --display_data=true
```

例：ペットボトルを所定位置に置く収録なら、NEW_DATASET_NAME = so101_bottle_place_2cam_100、EPISODES = 100、TASK_IN_ENGLISH = Pick up the plastic bottle and place it on the marked spot。これらは例なので自由に変更できます。

重要：収録後に Hugging Face 上で実際に作成された正確な repo_id を確認してください。日時が付いた別名で保存されている場合は、その完全な名前を次の学習コマンドに使います。既存のデータセットを意図せず上書きしないよう新しい名前を推奨します。

収録済みの別データを使う場合

収録をやり直す必要はありません。次ページの「18. 学習コマンド」で --dataset.repo_id だけを新しいデータの正確な ID に指定します。Hugging Face の非公開データはアクセス権とログイン状態も必要です。

18. ACT 学習の書き換えテンプレート

学習コマンドはここが最重要です。{{DATASET_REPO}} と {{MODEL_REPO}} は別の値です。

```
lerobot-train \
  --dataset.repo_id={{HF_USER}}/{{DATASET_REPO}} \
  --policy.type=act \
  --policy.device=mps \
  --policy.repo_id={{HF_USER}}/{{MODEL_REPO}} \
  --policy.push_to_hub=true \
  --output_dir=outputs/train/{{RUN_NAME}} \
  --job_name={{RUN_NAME}} \
  --batch_size={{BATCH_SIZE}} \
  --steps={{STEPS}} \
  --save_checkpoint=true \
  --save_freq={{SAVE_FREQ}} \
  --wandb.enable=false
```

置換する部分	例	意味
HF_USER	Pinset	Hugging Face ユーザー名
DATASET_REPO	so101_bottle_place_2cam_100	学習元データセットの名前（日時まで正確に）
MODEL_REPO	act_so101_bottle_2000	新しいモデル保存先
RUN_NAME	act_so101_bottle_2000	PC 上のフォルダ名 / ジョブ名
BATCH_SIZE	8	一度に処理するサンプル数
STEPS	2000	学習ステップ数
SAVE_FREQ	1000	チェックポイント保存間隔

実例：別データセットを 2000 steps で学習

```
lerobot-train \
  --dataset.repo_id=Pinset/so101_bottle_place_2cam_100 \
  --policy.type=act \
  --policy.device=mps \
  --policy.repo_id=Pinset/act_so101_bottle_2000 \
  --policy.push_to_hub=true \
  --output_dir=outputs/train/act_so101_bottle_2000 \
  --job_name=act_so101_bottle_2000 \
  --batch_size=8 \
  --steps=2000 \
  --save_checkpoint=true \
  --save_freq=1000 \
  --wandb.enable=false
```

※これは置換例です。実際のデータセット名やモデル名は存在確認が必要です。Mac のメモリ不足が起きるなら batch_size を下げて再実行します。出力先が既にある場合は別名を使用してください。

19. 学習済みモデルを切り替えて実機実行

「学習コマンドの `--policy.repo_id`」に設定したモデルを「実行コマンドの `--policy.path`」で読み込みます。

```
lerobot-rollout \
  --strategy.type=base \
  --robot.type=so101_follower \
  --robot.port={{FOLLOWER_PORT}} \
  --robot.id=my_awesome_follower_arm \
  --robot.cameras="{hand: {type: opencv, index_or_path: 0, width: 640, height: 480, fps: 30}, top: {type:
opencv, index_or_path: 1, width: 640, height: 480, fps: 30}}" \
  --policy.path={{HF_USER}}/{{MODEL_REPO}} \
  --policy.device=mps \
  --task="{{TASK_IN_ENGLISH}}" \
  --duration={{DURATION_SECONDS}} \
  --display_data=true
```

例： `policy.path=Pinset/act_so101_bottle_2000` に変更すると、ペットボトル用として学習させたモデルを読み込みます。 `task` は対応する作業説明に変更します。ただし ACT の実際の動作は学習データと観測画像に依存するため、 `task` 文字列だけを変えて新しい動作を覚えるわけではありません。

特に注意：2 カメラ構成の一致

データセットにある `observation.images.hand / observation.images.top` に対応する入力を、実機でも同じ名前・意味で渡してください。カメラが 0/1 で逆転していたら、 `index_or_path` を変更して映像と名前の対応を保ちます。別データセットのカメラ構成が違う場合、このテンプレートをそのまま使えるとは限りません。

実行前の確認手順

1. `conda activate lerobot` → `hf auth whoami` でアカウントを確認
2. `lerobot-find-port` → Follower / Leader のポートを確認
3. `lerobot-find-cameras opencv` → `hand / top` の映像を確認
4. Hugging Face で Dataset ID と Model ID の綴りを確認（404 防止）
5. 初回の rollout は周囲を空け、非常停止できる状態で短く試す

最後に：書き換える順番

新しい作業 → ①収録時の `dataset.repo_id` と `single_task` → ②学習時の `dataset.repo_id / policy.repo_id / output_dir / job_name` → ③実行時の `policy.path` と `task`。この流れで変更すると取り違えを減らせます。